



TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES
MULTIPLATAFORMA
Departamento de Informática

PROYECTO

KultuX
Manual Técnico

Autor/es: Cristo Macías Izaguirre, Sandra María Moñino García
Curso Académico: 2025/2026
Tutor Proyecto : María Mercedes Martínez Fragoso

Índice

1. Introducción	1
2. Objetivos	3
3. Tecnologías involucradas	4
3.1. Frontend	4
3.2. Backend	4
3.3. Base de datos	5
3.4. APIs externas	5
4. Proceso de desarrollo	6
4.1 Análisis	6
4.1.1 Diseño visual base	6
4.1.2 Diseño de la BBDD	8
4.1.2. 1 MODELO ENTIDAD - RELACIÓN	8
4.1.2. 2 MODELO RELACIONAL	9
4.1.3. Diagrama de casos de uso	17
4.2 Desarrollo	18
4.2.1. Diagrama de secuencias	18
4.2.2 Estructura del proyecto	19
Aplicación web — Angular	19
Backend — Spring Boot	20
Aplicación móvil — Flutter	21
Base de datos — PostgreSQL en Supabase	21
4.2.3 Aspectos destacables de la aplicación.	22
4.3 Pruebas realizadas	24
Pruebas del backend mediante Postman	24
Pruebas manuales	24
5. Proceso de despliegue	26
5. 1 Despliegue en Render:	26
5. 2 Despliegue en Vercel	30
5. 3 Despliegue de APK	32
6. Propuesta de mejora y trabajos futuros.	34
6.1 Mejoras de funcionalidades	34
6.2 Mejoras técnicas	34
7. Bibliografía	35

1. Introducción

KultuX es una plataforma digital orientada a la promoción, descubrimiento y gestión de actividades culturales, turísticas y de ocio en Extremadura. El proyecto nace con el objetivo de centralizar en un único entorno toda la información relacionada con eventos, alojamientos y establecimientos de restauración de la región, facilitando tanto a residentes como a turistas el acceso a planes, experiencias y servicios disponibles en distintas localidades extremeñas.

Actualmente, gran parte de la información sobre actividades y lugares de interés en Extremadura se encuentra dispersa entre redes sociales, páginas municipales, publicaciones temporales o plataformas poco especializadas. Esta situación dificulta la visibilidad de pequeños eventos y negocios locales, además de limitar la capacidad de los usuarios para descubrir actividades de manera rápida y organizada. KultuX surge como respuesta a esta necesidad, proporcionando una solución moderna, accesible y centralizada.

El proyecto se compone de tres bloques principales: una aplicación móvil destinada a los usuarios finales, una plataforma web de gestión orientada a administradores y gestores de contenido, y una landing page pública utilizada como punto de acceso e información general del ecosistema.

2. Objetivos

El objetivo principal de KultuX es crear un ecosistema digital capaz de conectar personas, actividades y negocios de Extremadura dentro de una misma plataforma, fomentando el turismo regional, la participación en eventos locales y la visibilidad de empresas relacionadas con el ocio y la hostelería.

De manera más concreta, los objetivos que se persiguen con el desarrollo de la plataforma son los siguientes:

- **Centralizar** en un único entorno la información sobre eventos, actividades, alojamientos y restaurantes de Extremadura.
- Facilitar el **descubrimiento** de planes y experiencias mediante herramientas intuitivas de búsqueda, filtrado y geolocalización.
- Ofrecer a gestores y empresas locales un espacio digital donde **administrar** y **promocionar** sus contenidos de forma autónoma.
- Dar **visibilidad** a actividades y negocios de pequeñas localidades, favoreciendo la promoción del entorno rural y del turismo local.
- Proporcionar una experiencia de usuario moderna, **accesible** desde dispositivos móviles y web.
- **Integrar** distintos tipos de contenido (actividades, alojamientos y restauración) en un único ecosistema digital.

3. Tecnologías involucradas

Tras el análisis de los requerimientos del sistema y la naturaleza multiplataforma del proyecto, se ha optado por un conjunto de tecnologías modernas y ampliamente utilizadas en el desarrollo profesional de aplicaciones. A continuación se detallan las principales herramientas y frameworks empleados.

3.1. Frontend

Aplicación móvil — Flutter: La aplicación móvil ha sido desarrollada con Flutter, el framework de Google para la creación de aplicaciones nativas multiplataforma a partir de una única base de código. La distribución se realiza mediante APK release para dispositivos Android.

Web de gestión — Angular: La plataforma de administración web ha sido implementada con Angular, framework de desarrollo frontend basado en TypeScript, orientado a la construcción de aplicaciones web robustas y escalables.

Despliegue frontend — Vercel: Tanto la web de gestión como la landing page son desplegadas a través de Vercel, plataforma de despliegue en la nube optimizada para aplicaciones frontend modernas.

3.2. Backend

Arquitectura de microservicios — Spring Boot: El backend está construido sobre una arquitectura de microservicios implementada con Spring Boot. Cada microservicio sigue una arquitectura interna por capas (controlador, servicio, repositorio), lo que favorece la separación de responsabilidades y la mantenibilidad del código.

Gestión del pool de conexiones — HikariCP: Cada microservicio gestiona las conexiones a la base de datos mediante HikariCP, el pool de conexiones integrado por defecto en Spring Boot. HikariCP permite reutilizar conexiones ya establecidas con la base de datos en lugar de crear una nueva por cada petición, reduciendo la latencia y optimizando el uso de recursos del sistema. Su configuración se define dentro de la capa de configuración de cada microservicio, estableciendo parámetros como el número máximo de conexiones simultáneas, el tiempo de espera y el tiempo de vida máximo de cada conexión.

API Gateway: Las llamadas procedentes de los frontends son gestionadas y orquestadas mediante un gateway centralizado, que actúa como punto de entrada único al sistema y se encarga de enrutar las peticiones hacia el microservicio correspondiente.

Despliegue backend — Render: Los microservicios son desplegados en Render, plataforma de hosting en la nube para aplicaciones backend. Con el fin de evitar la hibernación de los servicios en el plan gratuito, se han configurado llamadas automáticas periódicas mediante cronjob, garantizando así la disponibilidad continua del sistema.

3.3. Base de datos

PostgreSQL en Supabase: El almacenamiento de datos se realiza mediante PostgreSQL, alojado en Supabase. La base de datos se encuentra organizada en schemas independientes que simulan bases de datos separadas por dominio o microservicio, manteniendo así la coherencia con la arquitectura distribuida del sistema.

3.4. APIs externas

Brevo: Plataforma utilizada para la gestión y el envío de correos electrónicos transaccionales desde el sistema.

Cerocom: API empleada para la obtención de localidades de Extremadura, utilizada en la gestión geográfica de los contenidos de la plataforma.

4. Proceso de desarrollo

4.1 Análisis

4.1.1 Diseño visual base

Al inicio del proyecto se utilizó la herramienta Figma para la creación del diseño visual base de la aplicación. El objetivo de esta fase fue establecer una guía visual coherente antes de comenzar el desarrollo, definiendo la estructura de las pantallas principales, la paleta de colores, la tipografía y la distribución general de los elementos de la interfaz.

El diseño está basado en la identidad visual de KultuX, haciendo uso de una paleta de colores que combina el verde lima como color de acción principal, el blanco como fondo predominante y el negro para los textos y superficies secundarias. Esta combinación busca transmitir modernidad y dinamismo, adaptándose al carácter cultural y de ocio de la plataforma.

Las interfaces han sido diseñadas siguiendo un enfoque minimalista y funcional, priorizando la claridad visual y la facilidad de navegación. Los elementos interactivos resultan fácilmente reconocibles y la información se presenta de forma estructurada y jerarquizada.

A lo largo del proceso de implementación se han llevado a cabo ajustes y adaptaciones sobre el diseño original, motivados por las necesidades técnicas surgidas durante el desarrollo y por mejoras en la experiencia de usuario identificadas en fases posteriores.

Las pantallas diseñadas en Figma abarcan los principales flujos de la aplicación móvil: pantalla de bienvenida, registro e inicio de sesión, pantalla principal con listado de actividades, mapa interactivo de Extremadura, búsqueda con filtros, detalle de actividad, sección de establecimientos (restaurantes y alojamientos), perfil de usuario y pantalla de favoritos, entre otras.

Proyecto “Desarrollo de aplicaciones multiplataforma”

KultuX - Donde la cultura se encuentra contigo



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

A continuación se muestra el conjunto de pantallas que conformaron el diseño base creado para la aplicación:

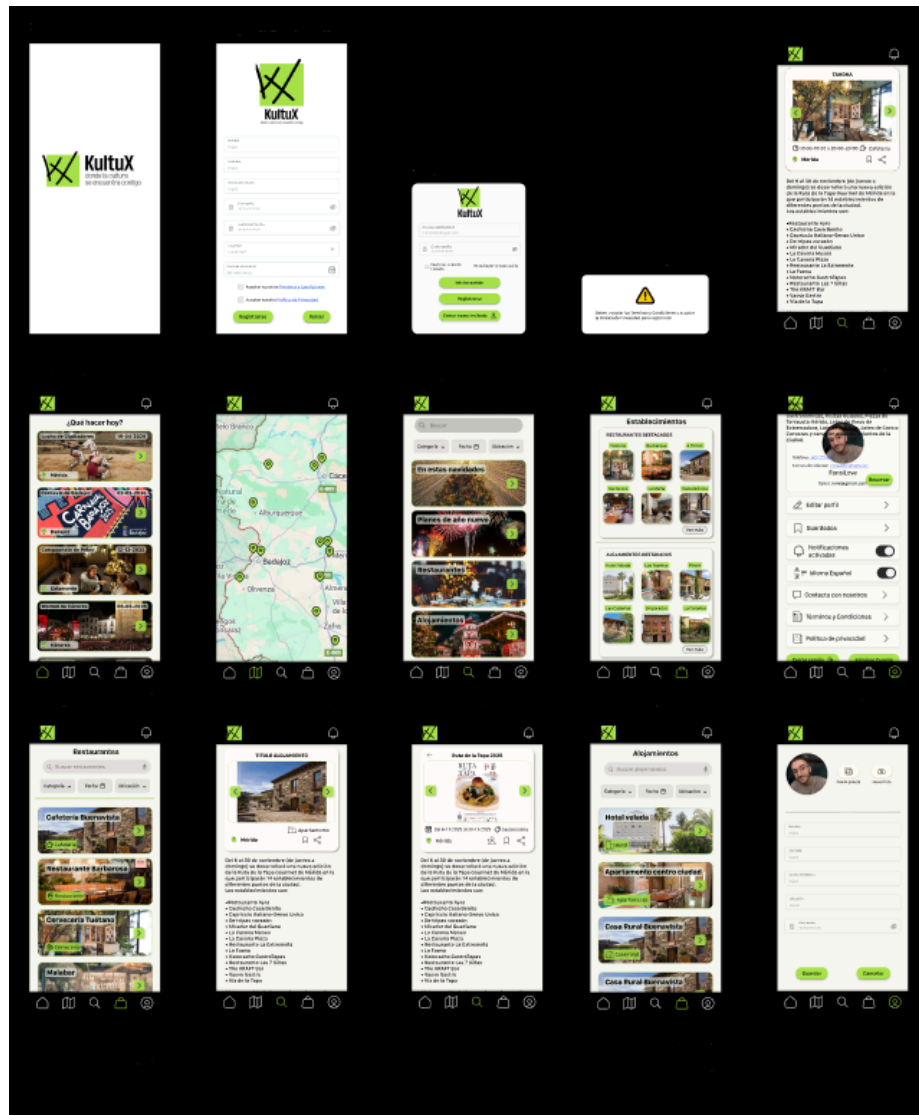


Imagen 1: Diseño Figma — Pantallas principales de la aplicación móvil KultuX



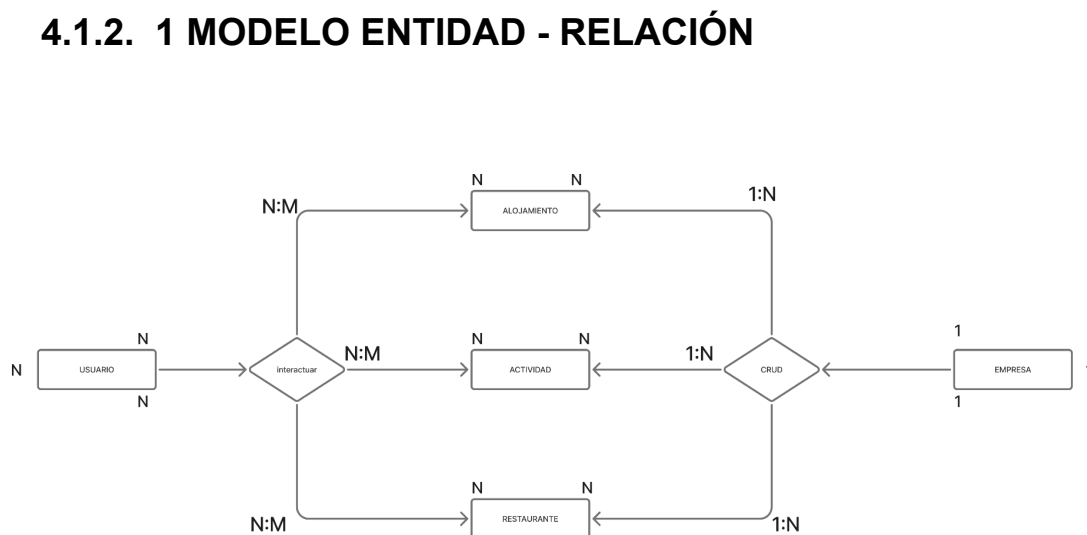
4.1.2 Diseño de la BBDD

Para el almacenamiento de los datos de la aplicación se ha optado por la utilización de PostgreSQL como sistema gestor de base de datos relacional, alojado en Supabase como plataforma de backend en la nube.

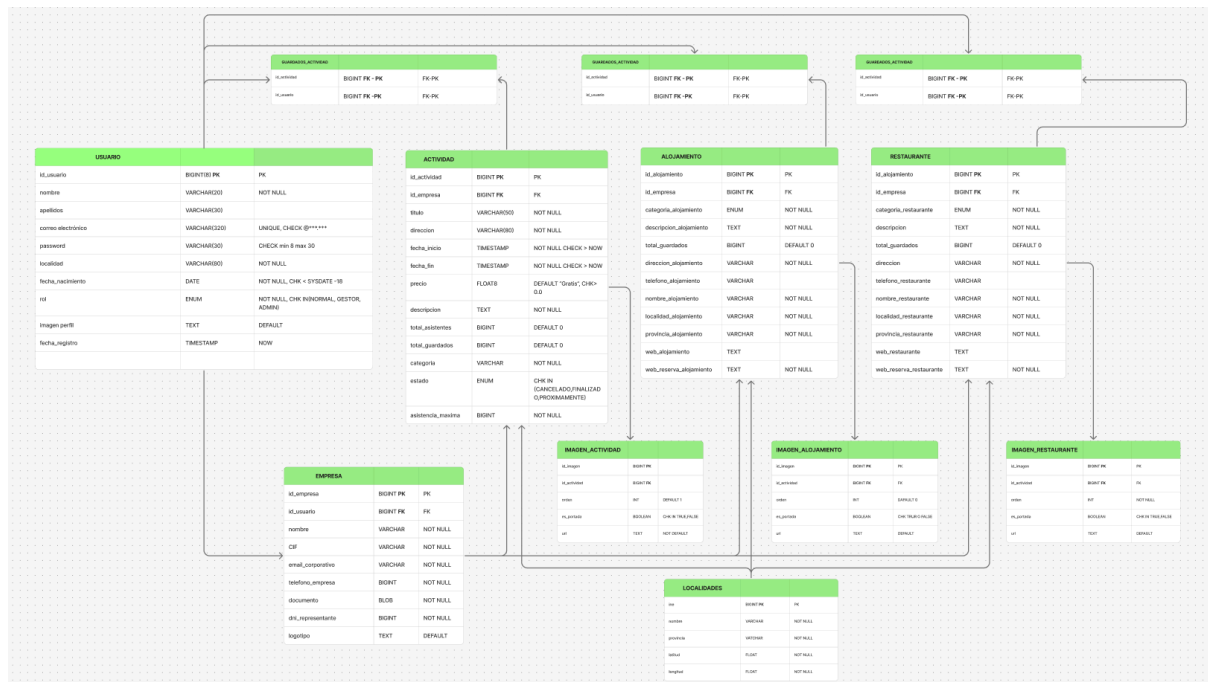
Con el fin de mantener la coherencia con la arquitectura de microservicios del backend, la base de datos se encuentra organizada mediante schemas independientes, cada uno de los cuales agrupa las tablas pertenecientes a un dominio funcional concreto. Los schemas definidos son: usuario, actividades, restaurantes, alojamientos, interaccion, y localidad. Esta separación lógica simula el comportamiento de bases de datos independientes por microservicio, facilitando la escalabilidad del sistema y el mantenimiento de cada módulo de forma aislada.

Las relaciones entre las distintas entidades se establecen mediante claves foráneas que referencian los identificadores de los registros correspondientes o, mediante el gateway, microservicio que orquesta las relaciones garantizando así la integridad referencial del conjunto del sistema.

A continuación se adjuntan los diagramas de tablas por schema, el diagrama relacional y el diagrama entidad-relación de la base de datos del proyecto:



4.1.2. 2 MODELO RELACIONAL



4.1.2. 3 MODELO DE DATOS

USUARIO

Nombre campo	Tipo de dato	LON.	DEC.
id_usuario	numérico	8	0
nombre	alfanumérico	20	
apellidos	alfanumérico	30	
email	alfanumérico	320	
password	alfanumérico	30	
localidad	alfanumérico	80	
fecha_nacimiento	fecha	10	
rol	alfanumérico	enum	
imagen_perfil	alfanumérico	url	
fecha_registro	fecha y hora	fecha	
id_ultima_empresa	numérico	8	0
dni	alfanumérico	9	

TABLA ACTIVIDAD

Nombre campo	Tipo de dato	LON.	DEC.
id_actividad	numérico	8	0
id_empresa	numérico	8	0
titulo	alfanumérico	50	
direccion	alfanumérico	80	
fecha_inicio	fecha y hora		
fecha_fin	fecha y hora		
precio	numérico	4	2
descripcion	alfanumérico	text	
total_asistentes	numérico	10	0
total_guardados	numérico	10	0
categoría	alfanumérico	enum	
estado	alfanumérico	enum	
asistencia_maxima	numérico	10	0
localidad	alfanumérico		
hora_inicio	fecha y hora		
hora_fin	fecha y hora		
creado_en	fecha		
url_compra	alfanumérico	url	
url_web	alfanumérico	url	

TABLA ALOJAMIENTO

Nombre campo	Tipo de dato	LON.	DEC.
id_alojamiento	numérico	8	0
id_empresa	numérico	8	0
categoria_alojamiento	alfanumérico	enum	
descripcion_alojamiento	alfanumérico	text	
total_guardados	numérico	8	0
direccion_alojamiento	alfanumérico	200	
telefono_alojamiento	alfanumérico	13	
nombre_alojamiento	alfanumérico	80	
localidad_alojamiento	alfanumérico	50	
web_alojamiento	alfanumérico	url	
web_reserva_alojamiento	alfanumérico	url	
email_alojamiento	alfanumérico	320	
estado_alojamiento	enum	30	
creado_en	fecha	fecha	

TABLA RESTAURANTE

Nombre campo	Tipo de dato	LON.	DEC.
id_restaurante	numérico	8	0
id_empresa	numérico	8	0
categoria_restaurante	alfanumérico	enum	
descripcion_restaurante	alfanumérico	text	
total_guardados	numérico	8	0
direccion	alfanumérico	200	
telefono_restaurante	alfanumérico	13	
nombre_restaurante	alfanumérico	80	
localidad_restaurante	alfanumérico	50	
web_restaurante	alfanumérico	url	
web_reserva_restaurante	alfanumérico	url	
horario	fecha y hora	fecha	
email	alfanumérico	320	
estado_restaurante	enum	40	
creado_en	fecha	fecha	

TABLA EMPRESA

Nombre campo	Tipo de dato	LON.	DEC.
id_empresa	numérico	8	0
id_usuario	numérico	8	0
nombre	alfanumérico	20	
CIF	alfanumérico	10	
email_corporativo	alfanumérico	250	
telefono_empresa	alfanumérico	12	
documento	url	url	
localidad	numérico	8	0
logotipo	alfanumérico	url	
direccion_empresa	alfanumérico	200	
creado_en	fecha	fecha	

TABLA IMAGEN_ACTIVIDAD

Nombre campo	Tipo de dato	LON.	DEC.
id_imagen	numérico	8	0
id_actividad	numérico	8	0
es_portada	booleano	true or false	
url	url		

TABLA IMAGEN_ALOJAMIENTO

Nombre campo	Tipo de dato	LON.	DEC.
id_imagen	numérico	8	0
id_alojamiento	numérico	8	0
es_portada	booleano	true or false	
url	url		

TABLA IMAGEN_RESTAURANTE

Nombre campo	Tipo de dato	LON.	DEC.
id_imagen	numérico	8	0
id_restaurante	numérico	8	0
es_portada	booleano	true or false	
url	url		

TABLA GUARDADOS-ACTIVIDAD

Nombre campo	Tipo de dato	LON.	DEC.
id_actividad	numérico	8	0
id_usuario	numérico	8	0
creado_en	fecha	fecha	

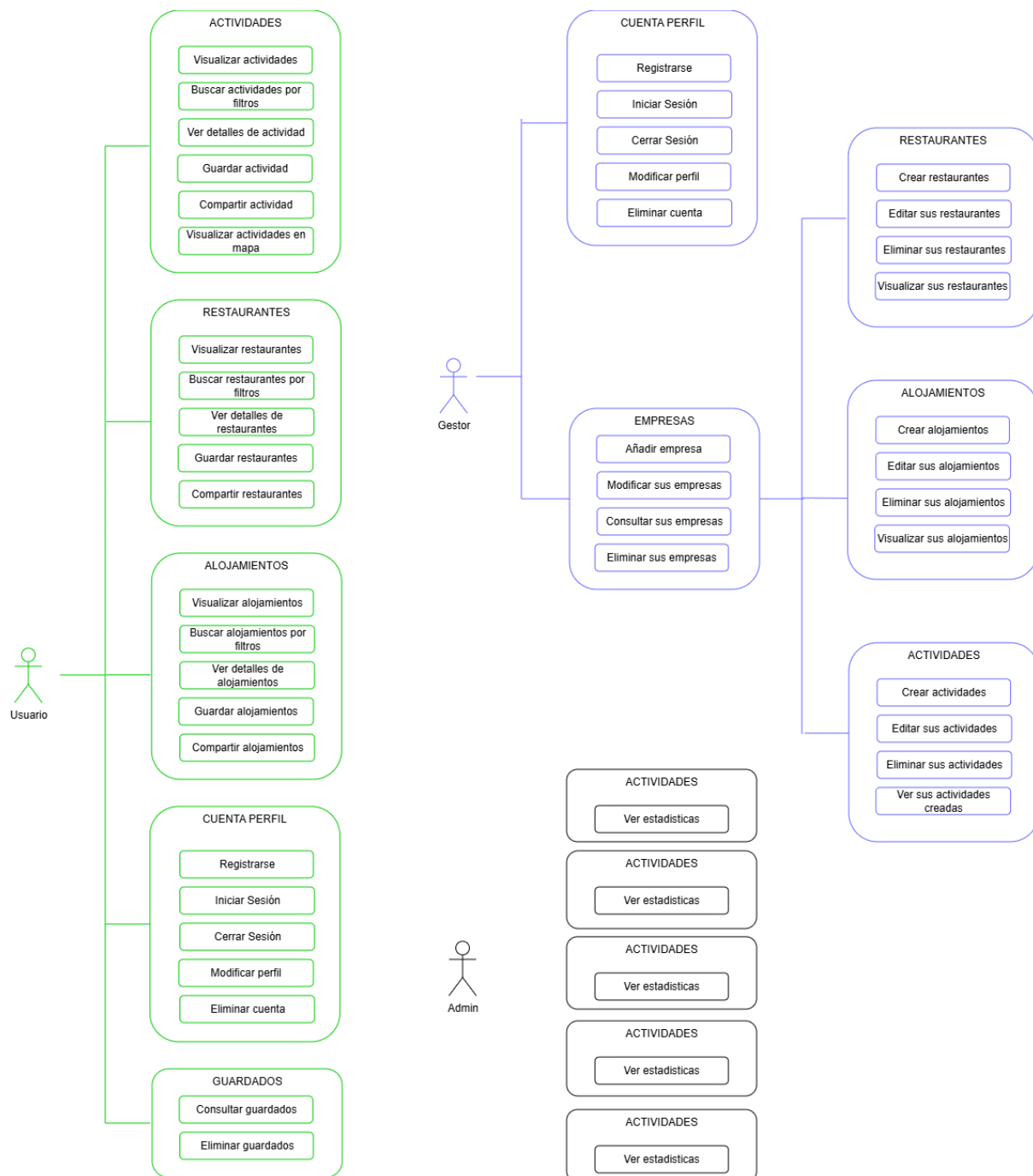
TABLA GUARDADOS-ALOJAMIENTOS

Nombre campo	Tipo de dato	LON.	DEC.
id_actividad	numérico	8	0
id_usuario	numérico	8	0
creado_en	fecha	fecha	

TABLA GUARDADOS-RESTAURANTES

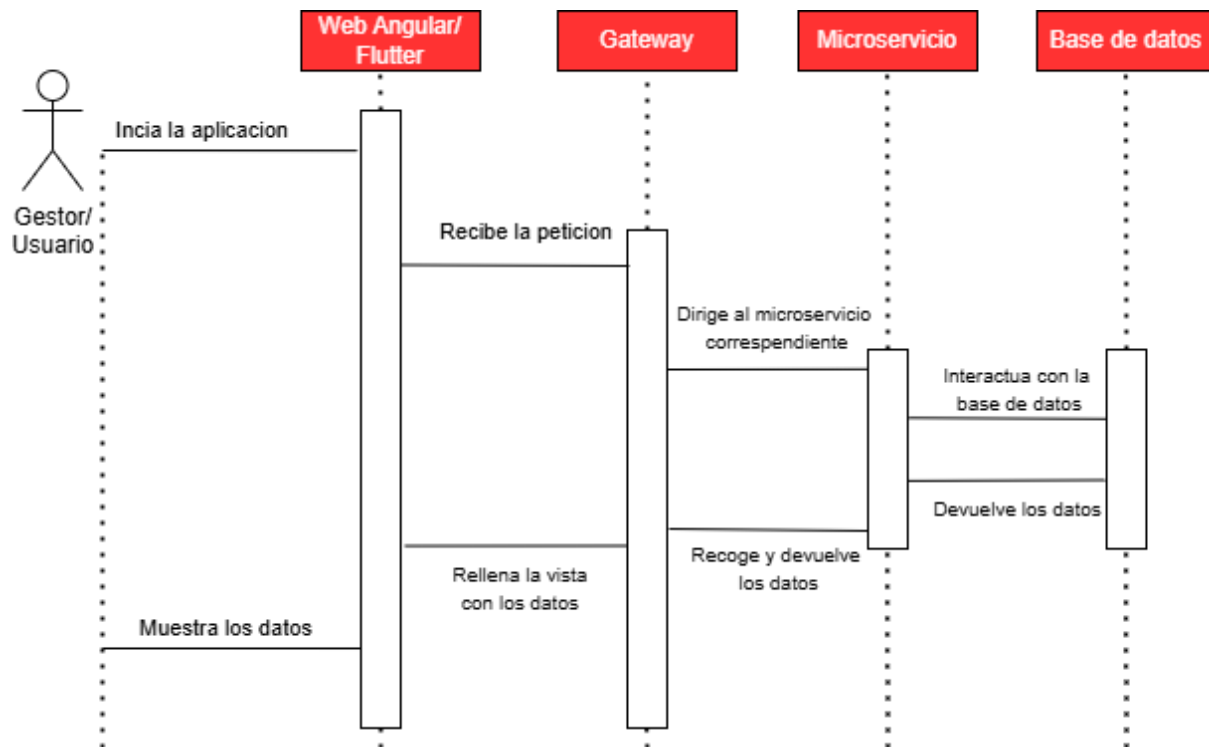
Nombre campo	Tipo de dato	LON.	DEC.
id_actividad	numérico	8	0
id_usuario	numérico	8	0
creado_en	fecha	fecha	

4.1.3. Diagrama de casos de uso

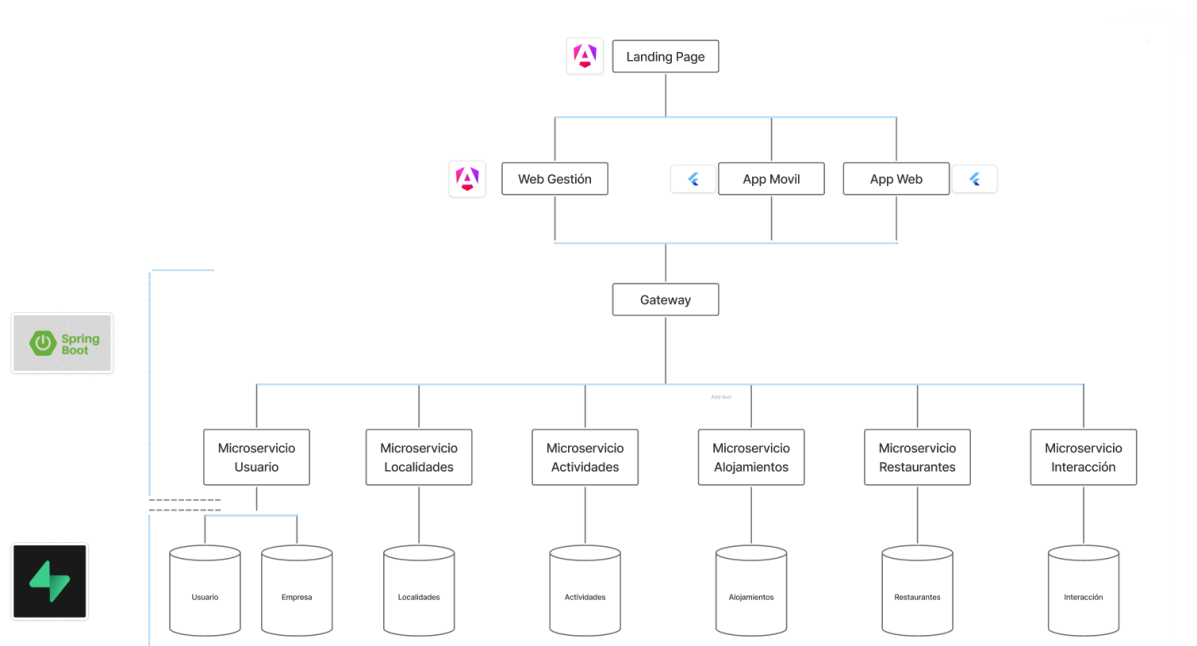


4.2 Desarrollo

4.2.1. Diagrama de secuencias



4.2.2 Estructura del proyecto



Aplicación web — Angular



La web de gestión está desarrollada siguiendo el patrón de **Single Page Application (SPA)** propio de Angular. La aplicación carga una única página base en la que el navbar y el footer permanecen estáticos, mientras que el contenido central se sustituye dinámicamente mediante el enrutador de Angular, que va renderizando los diferentes componentes según la navegación del usuario. Cada componente sigue la estructura estándar de Angular, compuesta por su archivo de plantilla **HTML**, su hoja de estilos **SCSS**, su fichero de

lógica **TypeScript** y su **fichero de pruebas**.

El proyecto contiene:

- **Pages:** Representan las diferentes vistas que se renderizan para el usuario, actuando cada una como una pantalla completa de la aplicación.
- **Components:** Elementos visuales reutilizables con una vista específica y su propia lógica interna, empleados para construir las páginas de forma modular.
- **Services:** Centralizan la comunicación con las APIs externas, gestionando las peticiones y devolviendo los datos procesados al resto de la aplicación.
- **Models:** Representan los objetos de dominio sobre los que trabaja la aplicación, adaptando y tipando los datos recibidos desde las APIs.
- **Guards:** Controlan el acceso a determinadas funcionalidades o vistas de la aplicación en función de condiciones específicas, como el estado de autenticación del usuario o su rol.
- **Pipes:** Son transformadores de datos que se aplican directamente en las plantillas HTML. Permiten modificar o formatear el valor de una variable antes de mostrarlo al usuario, sin alterar el dato original.

Backend — Spring Boot

Cada microservicio del backend está implementado con Spring Boot siguiendo una arquitectura por capas. Esta estructura separa claramente las responsabilidades del sistema en las siguientes capas: el **controlador**, encargado de recibir y gestionar las peticiones HTTP; el **servicio**, donde reside la lógica de negocio; el **repositorio**, responsable del acceso a los datos; y las **entidades**, que representan el modelo de datos.



Adicionalmente, cada microservicio cuenta con sus **DTOs** para la transferencia de datos entre capas y sus **clases de configuración**.

Aplicación móvil — Flutter

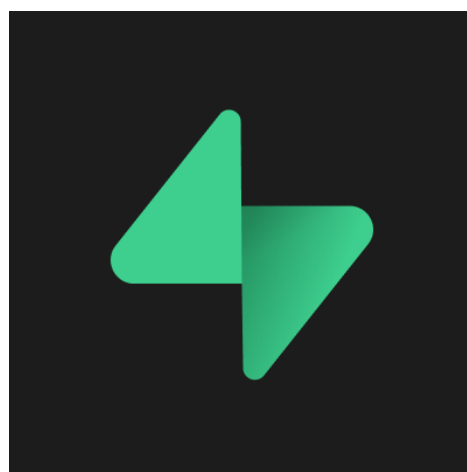


La aplicación móvil sigue un enfoque estructural equivalente al patrón SPA, donde el **Scaffold principal** actúa como contenedor base y el contenido central se reemplaza mediante componentes que simulan páginas completas. Junto a estos, existen **componentes** más específicos reutilizables, como botones o tarjetas. La lógica de comunicación con las APIs se centraliza en los **services**, mientras que los datos del usuario en sesión se persisten localmente mediante **SharedPreferences**, gestionados a través de un **repository**. Por último, los **modelos** representan los objetos de dominio con los que trabaja la aplicación.

Base de datos — PostgreSQL en Supabase

El almacenamiento de datos del sistema se gestiona mediante una única base de datos **PostgreSQL** alojada en **Supabase**. Con el fin de mantener la coherencia con la arquitectura de microservicios del backend, la base de datos se organiza internamente mediante schemas independientes, cada uno de los cuales agrupa las tablas pertenecientes a un dominio funcional concreto, simulando así el comportamiento de bases de datos separadas a las que cada microservicio accede de forma aislada.

Los schemas definidos son los siguientes:



Actividades: Contiene las tablas actividad e imagenes_actividad, almacenando la información de los eventos y actividades culturales y de ocio junto con su contenido multimedia asociado.

Usuario: Contiene las tablas usuario y empresa, gestionando tanto los datos de los usuarios registrados en la plataforma como la información de las empresas asociadas.

Alojamiento: Agrupa las tablas alojamiento e imagenes_alojamiento, almacenando la información de los establecimientos de alojamiento junto con su contenido multimedia.

Restaurante: Contiene las tablas restaurante e imagenes_restaurante, con los datos de los restaurantes y sus imágenes asociadas.

Localidades: Compuesto por la tabla localidades, que almacena la información geográfica de los municipios de Extremadura utilizados en la plataforma.

Interaccion: Agrupa las tablas guardados_actividades, guardados_restaurantes y guardados_alojamientos, registrando los contenidos que los usuarios guardan como favoritos dentro de la aplicación.

Adicionalmente, Supabase ofrece un sistema de almacenamiento de archivos mediante **buckets**, que se utiliza para gestionar el contenido multimedia de la plataforma. Los buckets definidos son: **logotipo_empresa**, **imagenes_actividades**, **imagenes_restaurantes**, **imagenes_alojamiento** e **imagenes_usuarios**, cada uno destinado a almacenar los recursos gráficos correspondientes a su dominio.

4.2.3 Aspectos destacables de la aplicación.

Como partes importantes a recalcar de la aplicación contiene ciertas funcionalidades clave o interesantes de explicar:

- **Mapa interactivo:** La aplicación móvil incorpora un mapa interactivo de Extremadura implementado mediante las librerías flutter_map y latlong2, complementadas con flutter_map_tile_caching para el almacenamiento en caché de los tiles del mapa. Las coordenadas de cada actividad se obtienen a partir de los datos proporcionados por la API de localidades, permitiendo representar geográficamente los eventos disponibles y facilitando al usuario la exploración visual del contenido por zonas.
- **Filtrado reactivo:** El sistema de filtrado de contenido en la aplicación web está implementado mediante Signals y Observables de Angular, lo que permite que los resultados se actualicen de forma reactiva en tiempo real conforme el usuario modifica los criterios de búsqueda, tras la búsqueda en la API.

El sistema de filtrado de flutter funciona por llamadas y con condiciones en los parámetros de la misma. La API devuelve los datos coincidentes y flutter los actualiza en la aplicación.

- **Sistema de favoritos:** Los usuarios pueden guardar actividades, restaurantes y alojamientos como contenido favorito. Cada acción de guardado realiza una llamada a la API correspondiente, que registra el elemento en la tabla asociada dentro del *schema* de interacción de la base de datos. De este modo, el contenido guardado queda vinculado al usuario y es accesible desde cualquier dispositivo.
- **Gestión de imágenes mediante buckets:** El contenido multimedia de la plataforma se gestiona a través de los buckets de almacenamiento de Supabase. El proceso consiste en subir la imagen al bucket correspondiente, obtener la URL pública generada por Supabase e insertarla en la tabla asociada de la base de datos. El frontend recibe directamente esta URL y la utiliza para mostrar las imágenes, evitando así la transferencia de archivos binarios a través de la API.
- **Recuperación de contraseña mediante email:** Para la recuperación de contraseña se ha integrado la plataforma Brevo, que gestiona el envío de correos electrónicos transaccionales. Cuando un usuario solicita restablecer su contraseña, el sistema genera un mensaje automatizado que es enviado a través de Brevo a la dirección de correo asociada a la cuenta.
- **Control de acceso por roles:** La plataforma diferencia tres tipos de usuario: usuario final, gestor y administrador. Los usuarios no gestores no tienen acceso a la web de gestión. Los gestores, por su parte, no pueden acceder a la aplicación móvil destinada al usuario final. Dentro de la web de gestión, el rol de administrador dispone de funcionalidades exclusivas inaccesibles para los gestores, lo que establece una jerarquía de permisos que garantiza la correcta separación de responsabilidades dentro del sistema.
- **Compartir actividades:** La aplicación móvil permite a los usuarios compartir actividades con otras personas a través de las opciones nativas del dispositivo, mediante la librería *share_plus*. Esta funcionalidad facilita la difusión del contenido de la plataforma fuera del entorno de la aplicación.
- **CronJob anti-hibernación:** Dado que los microservicios están desplegados en el plan gratuito de Render, estos pueden entrar en estado de hibernación tras un periodo de inactividad. Para evitarlo, se ha configurado un cronjob en cada microservicio que realiza llamadas automáticas periódicas, manteniendo los servicios activos y garantizando la disponibilidad continua del sistema.

4.3 Pruebas realizadas

Para la comprobación del correcto funcionamiento del sistema se han llevado a cabo pruebas en dos fases diferenciadas, adaptadas a la naturaleza de cada capa de la aplicación.

Pruebas del backend mediante Postman

Durante el desarrollo de cada microservicio, las pruebas se han realizado a través de **Postman**, herramienta que permite realizar peticiones HTTP directamente sobre los endpoints expuestos por cada servicio. Esto ha permitido verificar el correcto funcionamiento de cada endpoint de forma aislada, validando tanto las respuestas esperadas ante peticiones correctas como el comportamiento del sistema ante datos incorrectos o situaciones de error, antes de integrar los servicios con el frontend.

Pruebas manuales del sistema completo

Pruebas manuales

Una vez integrados los frontends con el backend, se han llevado a cabo pruebas **manuales** del sistema en su conjunto, generando casos de uso reales y comprobando el flujo completo de la aplicación. A continuación se detallan las pruebas principales realizadas y los problemas encontrados durante el proceso:

- **Gestión de contenido:** Se han realizado pruebas de creación, edición y eliminación de actividades, restaurantes y alojamientos, verificando que los cambios se reflejan correctamente tanto en la base de datos como en las vistas del frontend.
- **Sistema de favoritos:** Se ha comprobado el guardado y eliminación de contenido favorito por parte de los usuarios, validando que los registros se insertan y eliminan correctamente en las tablas correspondientes del schema de interacción.
- **Gestión de imágenes:** Se han realizado pruebas de subida de imágenes a los buckets de Supabase, verificando que las URLs generadas se almacenan correctamente en la base de datos y que las imágenes se muestran adecuadamente en los frontends.

- **Control de roles y acceso:** Se han verificado las restricciones de acceso para cada tipo de usuario, comprobando que los usuarios finales no acceden a la web de gestión, que los gestores no acceden a la aplicación móvil y que las funcionalidades exclusivas del administrador no son accesibles para roles inferiores.
- **Filtros y búsqueda:** Se han probado los distintos filtros disponibles en la aplicación, validando que los resultados se actualizan correctamente y que las combinaciones de filtros devuelven los datos esperados.
- **Verificación de estados vacíos:** Se han realizado pruebas con listas y datos vacíos para comprobar que la aplicación muestra mensajes informativos adecuados en lugar de vistas vacías sin contexto para el usuario.
- **Recuperación de contraseña:** Se ha verificado el flujo completo de recuperación de contraseña, comprobando que el correo se envía correctamente a través de Brevo y que el proceso de restablecimiento funciona de forma adecuada.
- **Eliminación de usuarios en cascada:** Se ha verificado el proceso de eliminación de un usuario y todos sus datos asociados. Esta operación se gestiona desde el gateway, que organiza y ejecuta una pila de llamadas secuenciales a los microservicios implicados, asegurando que todos los registros relacionados con el usuario (favoritos, imágenes, datos de perfil, etc.) son eliminados de forma ordenada y consistente antes de proceder a la eliminación de la cuenta en sí.

5. Proceso de despliegue

El despliegue del sistema se encuentra dividido en tres partes diferenciadas, correspondientes a cada una de las capas de la plataforma. Los microservicios del backend han sido desplegados en Render, mientras que los frontends web han sido publicados a través de Vercel. Por su parte, la aplicación móvil se distribuye mediante una APK release generada desde el entorno de desarrollo y publicada en GitHub Releases, desde donde puede ser descargada e instalada directamente en dispositivos Android, además de acceder a una url web desde que puede acceder cualquier dispositivo, dando con esto la facilidad a acceder desde IOs a una primera versión. A continuación se detalla el proceso seguido en cada caso.

5. 1 Despliegue en Render:

Servicios desplegados actualmente:

Production

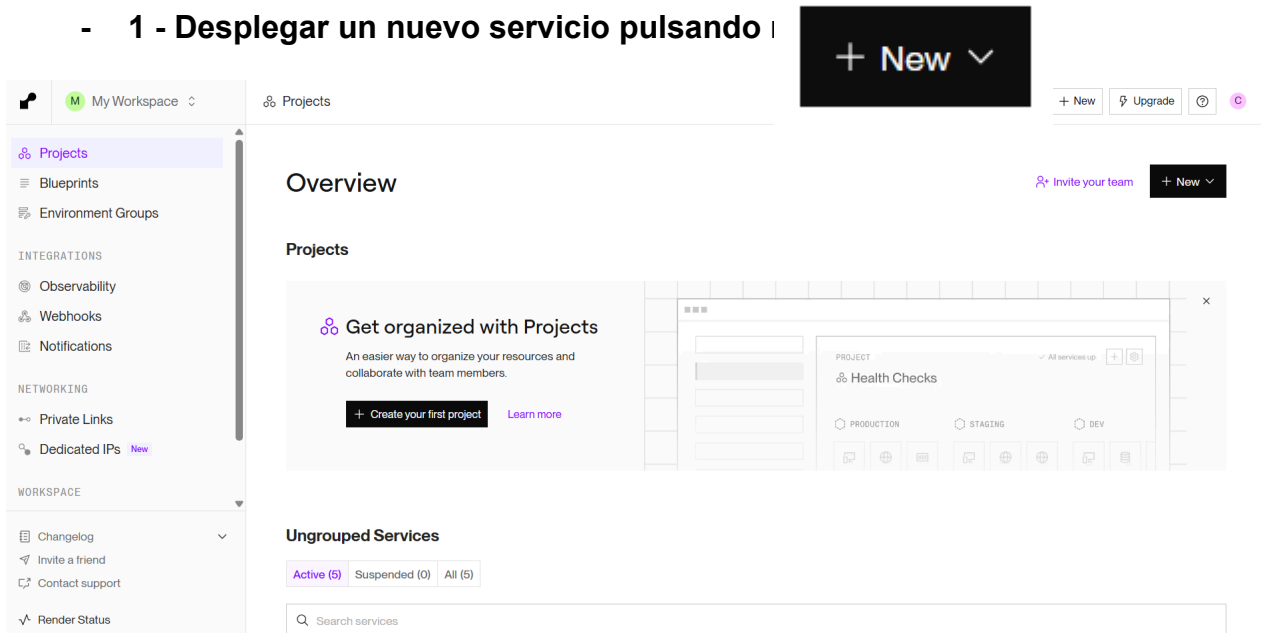
All (7) Services (7) Env Groups (0)

Q Search resources in Production

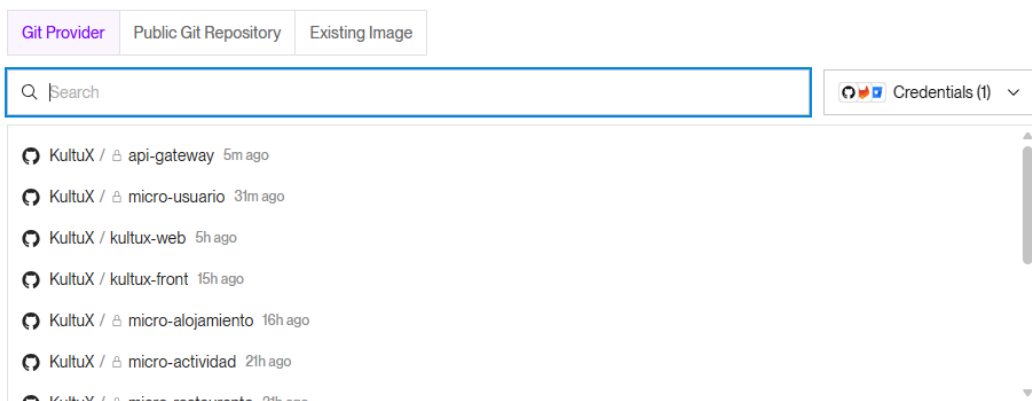
SERVICE NAME	STATUS	RUNTIME	REGION	UPDATED	
api-gateway	✓ Deployed	Docker	Oregon	2min	...
micro-usuario	✓ Deployed	Docker	Oregon	28min	...
micro-alojamiento	✓ Deployed	Docker	Oregon	16h	...
micro-interaccion	✓ Deployed	Docker	Oregon	16h	...
micro-restaurante	✓ Deployed	Docker	Oregon	16h	...
micro-actividad	✓ Deployed	Docker	Oregon	16h	...
micro-localidades	✓ Deployed	Docker	Oregon	16h	...
+ New service					

Paso a paso:

- **1 - Desplegar un nuevo servicio pulsando**



- **2 - Seleccionamos el repositorio que queremos desplegar**



- **3 - Aplicamos la configuración**, lo subiremos con docker, ya que tenemos un archivo **dockerfile** en el proyecto a nivel raíz, mostrado en una de las siguientes imágenes.

Se elegirá la rama utilizada del repositorio para desplegar y la región donde queremos que se lleve a cabo el despliegue.

Source Code KultuX / micro-actividad • 21h ago Edit

Name
A unique name for your web service.

Project Optional
Add this web service to a [project](#) once it's created.
My project / Production

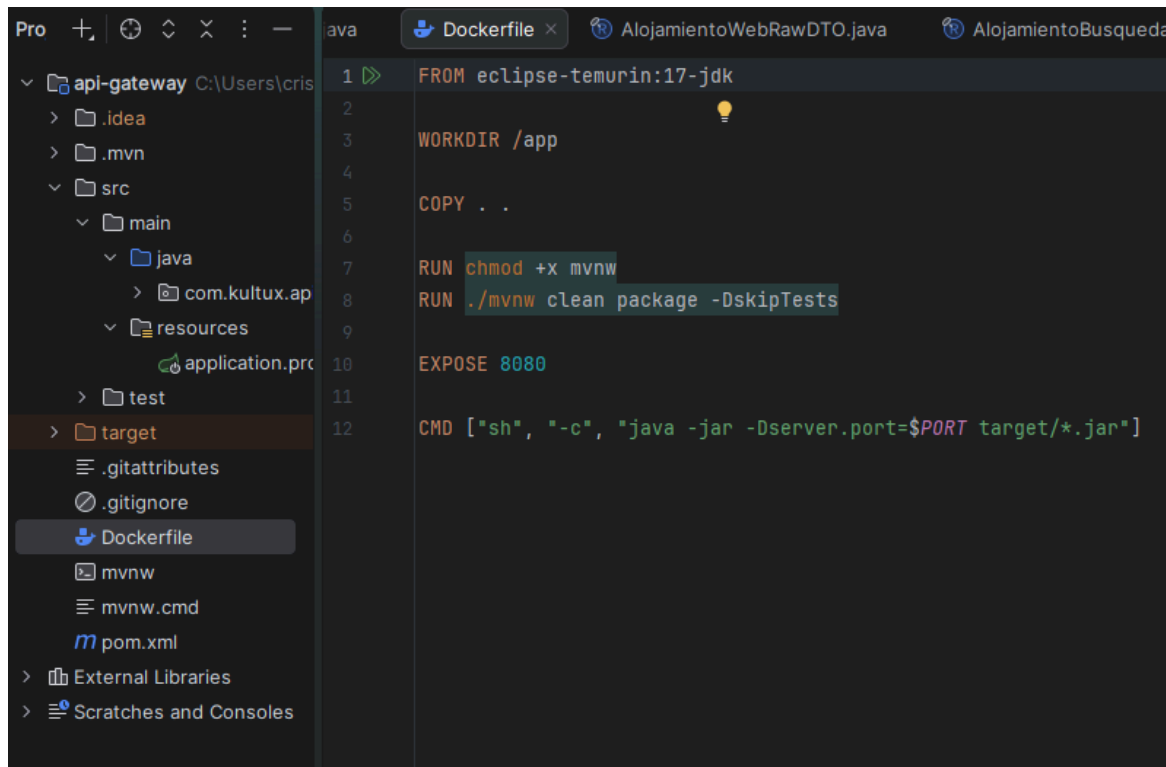
Language
Choose the [runtime environment](#) for this service.
Docker

Branch
The Git branch to build and deploy.
main

Region
Your services in the same [region](#) can communicate over a [private network](#). You currently have services running in [Oregon](#).
Oregon (US West) 7 existing services
[Deploy in a new region](#) +

Root Directory Optional
If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a [monorepo](#).

Archivo dockerfile



The screenshot shows an IDE with a project structure on the left and a Dockerfile being edited in the center. The project structure includes folders like .idea, .mvn, src, main, java, com.kultux.ap, resources, application.pro, test, target, .gitattributes, .gitignore, mvnw, mvnw.cmd, pom.xml, External Libraries, and Scratches and Consoles. The Dockerfile content is as follows:

```
1 FROM eclipse-temurin:17-jdk
2
3 WORKDIR /app
4
5 COPY . .
6
7 RUN chmod +x mvnw
8 RUN ./mvnw clean package -DskipTests
9
10 EXPOSE 8080
11
12 CMD ["sh", "-c", "java -jar -Dserver.port=$PORT target/*.jar"]
```

- 4 - Se aplica el plan que queremos para nuestro proyecto.

Instance Type

For hobby projects

Free \$0 / month	512 MB (RAM) 0.1 CPU
----------------------------	-------------------------

For professional use
For more power and to get the most out of Render, we recommend using one of our paid instance types. All paid instances support:

- Zero Downtime
- SSH Access
- Scaling
- One-off jobs
- Support for persistent disks

Upgrade to enable more features
Free instances spin down after periods of inactivity. They do not support SSH access, scaling, one-off jobs, or persistent disks. Select any paid instance type to enable these features.

Starter \$7 / month	512 MB (RAM) 0.5 CPU	Standard \$25 / month	2 GB (RAM) 1 CPU
Pro \$85 / month	4 GB (RAM) 2 CPU	Pro Plus \$175 / month	8 GB (RAM) 4 CPU
Pro Max \$225 / month	16 GB (RAM) 4 CPU	Pro Ultra \$450 / month	32 GB (RAM) 8 CPU

Need a [custom instance type](#)? We support up to 512 GB RAM and 64 CPUs.

- 5 - Configuramos las variables de entorno.

Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more](#).

KEY	VALUE		
ACTIVIDADES_STORAGE		👁	🗑
DATABASE_PASSWORD		👁	🗑
DATABASE_URL		👁	🗑
DATABASE_USER		👁	🗑
STORAGE_URL		👁	🗑
SUPABASE_KEY		👁	🗑

+ Add variable

▼

Save, rebuild, and deploy

▼

Cancel

- 6 - Desplegamos el proyecto.

Deploy Web Service

5. 2 Despliegue en Vercel

Paso a paso:

1 - Creación de fichero vercel.json

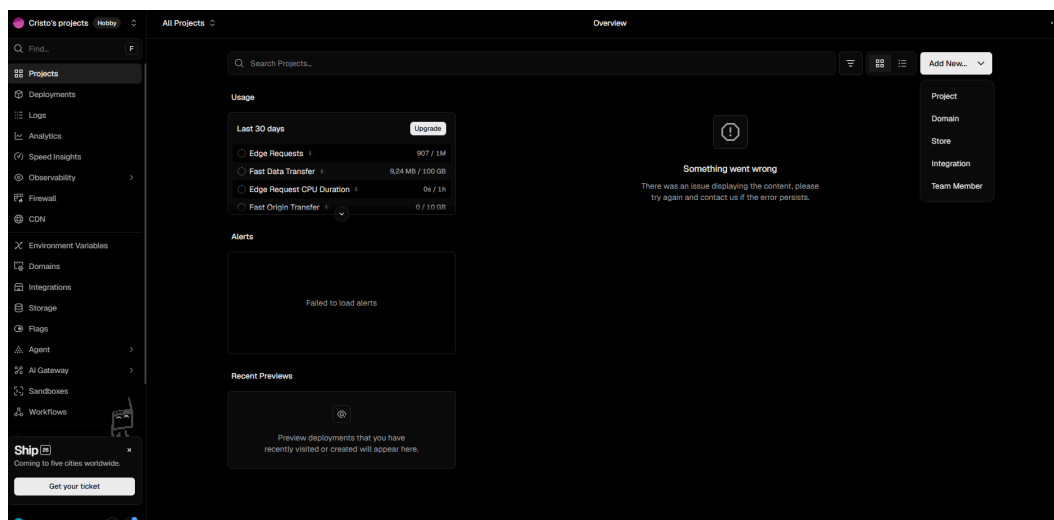
Se debe crear un fichero con los siguientes datos.



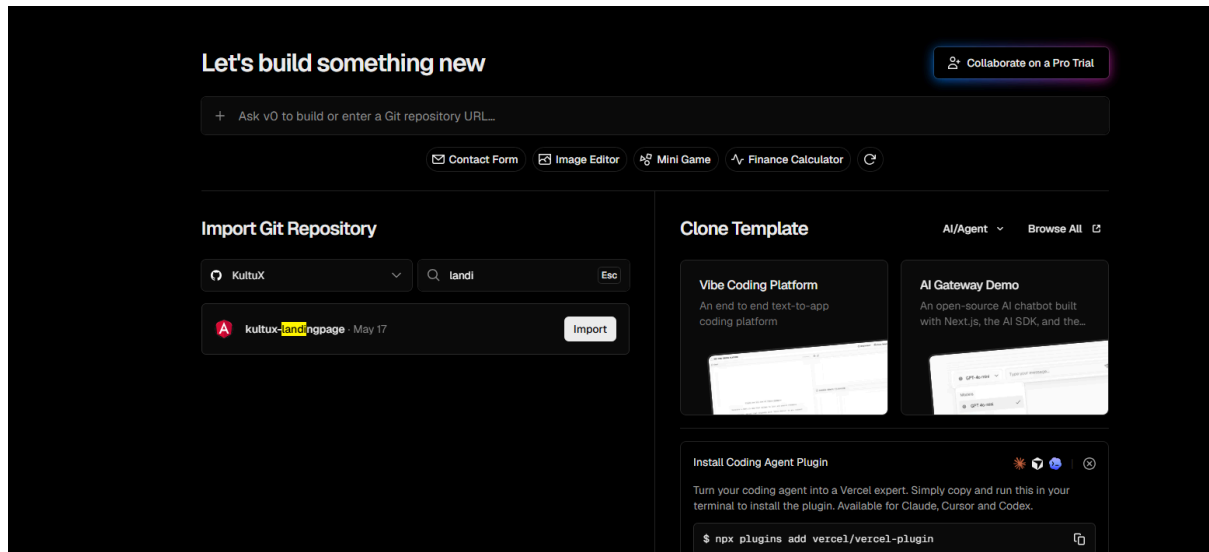
Que debe tener el siguiente contenido

```
{
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

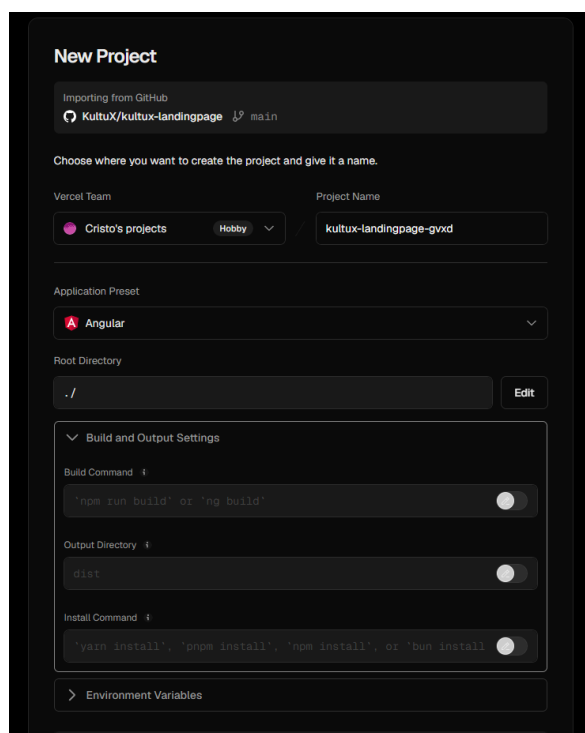
2 - Vamos a Vercel y pulsamos en Add New y seleccionamos Project.



3 - Continuamos importando el repositorio que vamos a desplegar



4 - Ajustamos el despliegue y le damos a continuar



Rellenamos con:

- **Build Command:** npm run build
- **Output Directory:** dist/kultux-landingpage/browser
- **Install Command:** npm install

Por último completamos con nuestras **variables de entorno**.

5 - Deploy

Por último pulsamos en deploy y el proyecto comenzará a desplegarse.

Deploy

5. 3 Despliegue de APK

Para realizar el despliegue de la apk seguiremos los siguientes pasos:

1 - Generación de la APK

En la consola del IDE en nuestro proyecto, ejecutamos los comandos:

- **flutter clean** : Limpia todas las dependencias del proyecto.
- **flutter pub get** : Recarga de nuevo todas las dependencias desde cero.
- **flutter build apk --release** : Genera la apk con todo nuevo y recién creado.

2 - Subida del APK a Github

Nos vamos a nuestro proyecto, al repositorio que queremos subir y pulsamos en la pestaña “Create a new release”.

The screenshot shows the GitHub interface for the repository 'kultux-front'. The repository is public and has 4 branches and 7 tags. The file list shows folders for different platforms: android, assets, build/web, ios, lib, linux, macos, and test. The 'Releases' section is highlighted with a red box, showing 7 tags and a link to 'Create a new release'.

3 - Configuración del release

Completamos el título, notas si las necesitamos, subimos el release y lo publicamos.

4 - Obtenemos el link del APK publicado.

En esta página, tenemos la información con la que hemos configurado nuestro apk release y está lista para ser probada.

6. Propuesta de mejora y trabajos futuros.

6.1 Mejoras de funcionalidades

Paquetes de eventos: Se plantea la incorporación de un sistema de agrupación de actividades bajo un evento contenedor, como por ejemplo una feria o festival. De este modo, varias actividades podrían quedar asociadas y presentadas de forma conjunta bajo un mismo nombre, mejorando la organización y la experiencia de descubrimiento del usuario.

Reseñas: Se plantea incorporar un sistema de reseñas sobre actividades, restaurantes y alojamientos que permita a los usuarios dejar comentarios, expresar su interés mediante un me gusta y otorgar una puntuación numérica o por estrellas. Esto fomentará la participación de la comunidad y aportará información de valor para otros usuarios a la hora de descubrir y elegir contenido dentro de la plataforma.

Notificaciones push y recomendaciones: Se incorporará un sistema de notificaciones que informe a los usuarios sobre actividades cercanas a su ubicación y sobre modificaciones en aquellas a las que tienen previsto asistir. Complementariamente, a partir del historial de contenido guardado y visualizado, se implementará un motor de recomendaciones que sugiera contenido adaptado a los intereses y ubicación de cada usuario.

6.2 Mejoras técnicas

Autenticación con JWT: Se plantea implementar un sistema de seguridad basado en tokens JWT para la autenticación y autorización de usuarios. Esto permitirá proteger los endpoints de la API, gestionar las sesiones de forma segura y controlar el acceso a los recursos según el rol del usuario, eliminando la dependencia actual de mecanismos menos robustos.

Caché de peticiones: Para mejorar el rendimiento del sistema y reducir la carga sobre los microservicios, se podría implementar una capa de caché que almacene temporalmente las respuestas de las peticiones más frecuentes.

Migración a un plan de hosting de pago: Con el crecimiento de la plataforma, se contempla la migración de los servicios actualmente desplegados en el plan gratuito de Render a un entorno de producción más estable, eliminando la necesidad del cronjob anti-hibernación y garantizando tiempos de respuesta óptimos.

7. Bibliografía

- **Flutter** — Framework principal para el desarrollo de la aplicación móvil: <https://flutter.dev>
- **Angular** — Framework para el desarrollo de la aplicación web de gestión y la landing page: <https://angular.dev>
- **Spring Boot** — Framework para el desarrollo de los microservicios del backend: <https://spring.io/projects/spring-boot>
- **Supabase** — Plataforma de base de datos y almacenamiento en la nube: <https://supabase.com>
- **Render** — Plataforma de despliegue del backend: <https://render.com>
- **Vercel** — Plataforma de despliegue del frontend: <https://vercel.com>
- **Brevo** — Plataforma para el envío de correos electrónicos transaccionales: <https://www.brevo.com>
- **Cerocom** — API para la obtención de localidades de Extremadura: <https://api.cerocom.net/>
- **Postman** — Herramienta utilizada para las pruebas de los endpoints del backend: <https://www.postman.com>
- **Figma** — Herramienta utilizada para el diseño visual base de la aplicación: <https://www.figma.com>
- **GitHub** — Plataforma de control de versiones y publicación de releases: <https://github.com>
- **flutter_map** — Librería Flutter para la implementación del mapa interactivo: https://pub.dev/packages/flutter_map
- **latlong2** — Librería Flutter para la gestión de coordenadas geográficas: <https://pub.dev/packages/latlong2>
- **share_plus** — Librería Flutter para compartir contenido desde la aplicación: https://pub.dev/packages/share_plus